Operators
&
Type Conversion

By PrudviKrishna

Python Operators

Operators in Python

- **Arithmetic**: '+', '-', '*', '/', '%', '**', '//'
- **Comparison**: '==', '!=', '>', '<', '>=', '<='
- **Logical**: 'and', 'or', 'not'
- **Bitwise**: '&', '|', '^', '~', '<<', '>>'
- **Assignment**: '=', '+=', '-=', '*=', '/=', '%=', '**=', '//='
- **Identity**: 'is', 'is not'

Membership: 'in', ‘not in’

Note: For more clear understanding of each operator, let’s look at code examples

Type Conversion

In Python, **type conversion** refers to converting one data type to another.

There are two types of conversions:

Implicit Type Conversion: The Python interpreter automatically converts one data type to another.

Explicit Type Conversion (Type Casting): The programmer manually converts the data type using specific functions.

1. Implicit Type Conversion

Python automatically converts a smaller data type to a larger one during operations where necessary. This process is also known as **coercion**.

Example Code:

```
x = 10      # Integer
y = 2.5     # Float

result = x + y   # Python automatically converts `x` to float
print(result)    # Output: 12.5
print(type(result))  # Output: <class 'float'>
```

Type Conversion

2. Explicit Type Conversion (Type Casting)

Explicit type conversion is done by using built-in functions. The most commonly used type casting functions are:

int() – Converts to integer.
float() – Converts to float.
str() – Converts to string.
list() – Converts to list.
tuple() – Converts to tuple.
set() – Converts to set.
dict() – Converts to dictionary.

Important Considerations:
If you try to convert a string that cannot be interpreted as a number
Ex: int('abc'), it will raise a ValueError.

Type casting should be used carefully when dealing with incompatible data types.

Thank you